Encrypted Cloud Storage with Searchable Symmetric Encryption

Vijayakumar P, Professor, SENSE

Shravan Adash M, 23BEC1149

Preetish RK, 23BEC1470

Prajinesh P, 23BEC1113

Abstract: Storing files with a cloud provider is cheap and convenient, but it means handing sensitive data to infrastructure that its owner does not control. Encrypting files on the client side solves the confidentiality problem, yet a server that holds only ciphertext cannot run a keyword search, so the user ends up downloading and decrypting the whole collection just to find one file. This project addresses that trade-off using Searchable Symmetric Encryption (SSE). We built a storage system split into two operating-system processes: a trusted client that owns all key material, and an untrusted cloud service that handles nothing but ciphertext. The client derives keys from the user's password with Argon2id and separates them using HKDF-SHA256, encrypts both file contents and filenames with AES-256-GCM, and builds a static encrypted inverted index with HMAC-SHA256 based on the basic construction of Cash et al. [13]. Given an opaque token, the cloud can locate matching entries in this index while learning neither the keyword nor the documents. Beyond single-keyword search, the implementation handles conjunctive (AND) queries, document deletion, and password rotation through full re-encryption. A reproducible benchmark suite measures cryptographic cost, index construction time, search latency, index rebuild cost and storage overhead. The leakage that remains, namely the search pattern, the access pattern and the size of the index, is shown directly in traffic captured between the client and the cloud.

Keywords: Searchable symmetric encryption (SSE); encrypted cloud storage; encrypted inverted index; AES-256-GCM; HMAC-SHA256; Argon2id; leakage analysis

I. INTRODUCTION

Many people and organisations now rely on third-party cloud services to hold their documents, mainly because these services scale easily, stay available and remove the need to run local storage hardware. Once data sits on someone else's servers, its owner has to trust whoever operates them. Anything stored in readable form is exposed to a provider that is curious or has been compromised, to a malicious insider, and to any attacker who manages to obtain a copy of the storage.

Client-side encryption with a standard cipher such as the Advanced Encryption Standard (AES) keeps the contents secret, but it also takes away the server's ability to help with retrieval. A ciphertext gives no hint of which words the original file contained, so the server cannot answer a request such as "which of my files mention this word?". The only option left to the user is to fetch everything, decrypt it locally and search it there, and the cost of doing so rises directly with the size of the collection.

SSE was developed to close this gap. Before uploading, the owner encrypts the files and also prepares an encrypted index over their keywords. When the owner later wants to search, a secret key and the chosen keyword are combined into a search token. The server applies this token to the index and returns the encrypted results without ever seeing the keyword or any plaintext. Turning this idea into a usable system raises its own questions: how to build the index safely, how to generate tokens cheaply, how to keep queries fast, how to handle changes to the stored data, and how to account for the information that efficient SSE schemes inevitably expose through their search and access patterns.

Much of the recent SSE literature [1]–[12] extends the schemes themselves, adding support for dynamic updates, access control, queries over several keywords, forward and backward privacy, deletion and more scalable index structures. These schemes are usually evaluated by analysing or timing their cryptographic operations rather than by running them inside a complete application. The aim of our project is more modest and practical. We built a working encrypted storage application in which the trusted client and

the untrusted cloud are genuinely separate processes, benchmarked it end to end in a repeatable way, and used recorded network traffic to show what the server can actually observe.

The remainder of the report is arranged as follows. Section II discusses recent journal work on searchable symmetric encryption. Section III explains the architecture of our system, its cryptographic modules and the complete algorithm. Section IV reports the implementation results and the performance measurements, and Section V summarises our observations and outlines future work.

II. RELATED WORK

Access control in shared settings is the focus of two of the papers we reviewed. SEAC, by Hu et al. [1], targets the cloud-assisted sharing of electronic protected health information. Its secure indexes carry permission information, which lets the scheme decide which clients may search and which may update the encrypted records, while keeping this permission check lightweight. The paper is a useful reminder that encrypted search alone is not enough once several parties share data. Our project sits at the other end of this spectrum: there is one data owner, so we have no permission layer, and our attention went instead to making sure the client–server trust boundary actually holds in a running program, a question that [1] does not examine. Yang et al. [3] approach access control from a different direction. They point out that many dynamic SSE schemes can restrict who searches by keyword or who modifies the encrypted data, but rarely both together, and they design a scheme for multi-user cloud computing that covers both operations. Because our vault has a single owner, cross-user authorisation of this kind is outside our design, and we treat it as a possible extension.

Xu et al. [2] adopt a stronger threat model in which the cloud may be actively malicious, which they describe as a zero-trust environment. Their dynamic scheme provides forward security, supports several users and lets each user check the results returned by the server, relying on oblivious pseudorandom functions and hash-based state structures to protect both queries and updates. We agree that the search process itself needs protection, but our implementation does not give the client a way to verify that returned results are complete, and it has no multi-user machinery. For a first implementation we preferred a design in which every security property could be explained and checked with tests.

Two papers deal with queries that combine keywords. The scheme in [4] provides forward and backward privacy for conjunctive dynamic SSE. A three-dimensional chain structure lets it evaluate several keywords together, and the authors refine the leakage function so that searches and updates reveal less, while also reporting much lower communication cost for multi-keyword queries. Our own conjunctive search is far simpler in comparison. The client runs one single-keyword search per term and intersects the results itself. No change to the index or the server was needed, but the server learns more than it would under [4], as Section III explains. Ren et al. [5] also make conjunctive queries efficient, in their case with a cuckoo filter, and add non-interactive real deletion by keeping separate search databases so that deleted ciphertexts can be physically removed. Deletion was something we also had to handle. In our system, removing a document means first rebuilding the encrypted index without it and only then deleting its ciphertext. This is straightforward and correct for a static index, but the work involved scales with the number of documents in the vault.

How the encrypted index is organised has a large effect on performance. Zhang et al. [6] pair a B+-tree reverse index with Bloom filters, which speeds up both index construction and search while still allowing dynamic updates, and they show that the choice of index structure becomes decisive as datasets grow. Their evaluation concentrates on the index itself and does not capture the delay a user perceives once the server sits across a network. In our own measurements (Section IV), network round trips rather than index lookups accounted for most of the user-visible delay. BloomSec, proposed by Khan et al. [10], stores a separate Bloom filter for each file in place of a single combined keyword index, which reduces the communication needed to transmit large lookup structures. Its central point, that the representation of the index shapes both search speed and communication cost, lines up with what we saw when measuring the storage taken by our encrypted index.

Other work broadens the kinds of queries and users that SSE can support. Wang et al. [7] remove the key-escrow problem from multi-client SSE through a decentralised design and support Boolean keyword queries with sub-linear search complexity, testing their system on the Enron dataset. MMKFB, by Wu et al. [8], allows multiple clients to run multi-keyword threshold queries with forward and backward privacy; additive

homomorphism is its main tool, and the scheme is particularly efficient when many files change in one batch. Xue et al. [11] work in a peer-to-peer community setting, where they combine ranked multi-keyword search, fine-grained access control with hidden attributes, verifiable results and a points-based incentive for participants. Each of these designs needs considerably more cryptography than a basic single-keyword scheme, and all of them address multi-client or multi-party scenarios that our single-owner vault does not. Our project goes the other way and keeps the scheme small enough that the source of every measured cost can be identified.

Leakage is where the gap between our system and the current state of the art is clearest. Wu et al. [9] present a verifiable searchable encryption scheme that hides both the search pattern and the access pattern. Polynomial evaluations are masked with random values, which avoids costly multiplications between ciphertexts, and users can confirm that the results they receive are correct and complete. The search and access patterns are exactly the two leakages that our implementation still has. We list hiding them as future work and make no claim to have done so.

Mohammed et al. [12] survey roughly eighty SSE schemes, grouping them by the search functionality they offer (single-keyword, multi-keyword, fuzzy, ranked, semantic and dynamic) and identifying privacy gaps that current implementations still leave open. Their classification places single-keyword static SSE as the base on which the richer variants are built. That is the class our system belongs to: its encrypted index follows Π_{bas}, the basic static construction of Cash et al. [13].

Taken together, the 2025–2026 papers reviewed above mostly improve the SSE scheme itself. Typical additions are support for updates, forward and backward privacy, authorisation across clients, Boolean or conjunctive queries, result verification and improved index structures, and they judge their contributions mainly by the cost of the cryptography and data structures involved. Three practical questions receive less coverage: whether the separation between data owner and server is really enforced in a running program, where the end-to-end time goes once network requests are counted, and what the leakage looks like in actual traffic as opposed to a formal leakage function. Our project concentrates on these questions. The owner and the server run as separate processes, the costs of cryptography, indexing, search, rebuilds and storage are measured with a reproducible benchmark suite, and recorded client–cloud traffic is used to show what the untrusted cloud can and cannot see.

III. PROPOSED SSE-BASED ENCRYPTED CLOUD STORAGE SYSTEM

The system is meant for a single authorised user who wants to keep documents on a server they do not trust and still be able to find those documents by keyword. The server should learn nothing about the contents of the files, their names or the words being searched for. Two operating-system processes make up the implementation. The trusted client listens on port 5000; it keeps every key and carries out all of the cryptography. The untrusted cloud listens on port 5001, and its job is limited to storing ciphertext and responding to search requests. The user's web browser connects only to the client and never receives any key.

Unlocking the vault starts with Argon2id, which turns the password into a master key; HKDF-SHA256 then splits that master key into one key for file encryption and one for the index. An uploaded file is converted to text, broken into a set of keywords, and encrypted with AES-256-GCM along with its filename. The client then produces an encrypted inverted index, using HMAC-SHA256 as the pseudorandom function and following Π_{bas} from Cash et al. [13], and swaps it into the cloud in one atomic step. A search sends the cloud a token for each keyword and nothing else. From that token the cloud recomputes index labels and returns the encrypted identifiers it finds, which the client then decrypts. Queries with several keywords are answered by intersecting the single-keyword results on the client. The subsections below describe each part: key management, authenticated encryption, index construction, token-based search, deletion, password rotation and the leakage model.

3.1 Key Derivation and Vault Unlock

Since all stored data ultimately depends on the keys, the password itself is never used to encrypt anything. Creating a new vault produces a random 16-byte salt, which is kept on the cloud. Each time the user unlocks the vault, the client feeds the password and this salt into Argon2id. Because Argon2id is memory-

hard, guessing passwords at scale on GPUs or custom hardware becomes expensive. We set its time cost to 3, its memory cost to 64 MiB and its parallelism to 4, following RFC 9106 [14]. The 256-bit output, the master key, is only an intermediate value. HKDF-SHA256 derives two separate sub-keys from it under distinct, versioned labels, so that learning one sub-key gives an attacker no information about the other:

$$MK = Argon2id(pw, salt;\ t = 3,\ m = 64\ MiB,\ p = 4) \tag{1}$$

$$K_{file} = HKDF(MK, \text{“vault:k_file:v1”}),\quad K_{index} = HKDF(MK, \text{“vault:k_index:v1”}) \tag{2}$$

Here pw denotes the password, salt the 16-byte random value stored on the cloud and MK the 256-bit master key. K_{file} protects document contents and filenames, while K_{index} is the root from which every keyword key is derived.

Without an explicit check, a wrong password would go unnoticed, because searching the index with incorrectly derived keys fails silently instead of raising an error. We therefore keep a password verifier V on the cloud next to the salt. V is a fixed constant encrypted with AES-GCM under K_{index}. The client accepts the derived keys only after decrypting V successfully, so an incorrect password is caught before any key is passed to the upload or indexing code.

$$V = AES\text{-}GCM(K_{index}, \text{“vault-verifier:v1”}) \tag{3}$$

Step 1 – Master Key Derivation: The client retrieves the salt, which is not secret, from the cloud and runs Argon2id with the parameters given above. On the machine used for evaluation, one unlock takes about 57 ms. A legitimate user barely notices this, yet an attacker pays the same cost for every password guess.

Step 2 – Sub-Key Separation and Password Verification: HKDF-SHA256 produces K_{file} and K_{index} from the master key, and the client attempts to decrypt the verifier using K_{index}. If authentication fails, the unlock request is refused with an error and no key is retained. On success, both sub-keys live in the trusted client’s process memory until the session ends, and are never written anywhere else.

3.2 Document Encryption Module

For document encryption we chose AES-256-GCM, an authenticated encryption mode that protects both confidentiality and integrity in a single operation. Every call draws a new 96-bit nonce from the operating system’s secure random number generator. The encryption function has no parameter through which a caller could pass its own nonce, so the nonce reuse that would break GCM cannot happen by accident. Any modification to a stored ciphertext is caught by the 128-bit authentication tag. Whether decryption fails because of a wrong key or because bytes were altered, the client reports the same generic error, which avoids giving an attacker an oracle-style side channel. Contents and filename are encrypted as two separate ciphertexts, and the cloud has access to neither in readable form.

$$C = AES\text{-}GCM(K_{file}, N, M) = (N, CT, T) \tag{4}$$

In this notation M is the plaintext (either a document or a filename), N is the random 96-bit nonce, CT is the resulting ciphertext and T is the 128-bit tag. The 12-byte nonce and 16-byte tag make every stored ciphertext exactly 28 bytes larger than its plaintext, and our measurements in Section IV confirm this.

Step 1 – Text Extraction and Identifier Assignment: Accepted inputs are UTF-8 text, PDFs that contain a readable text layer, and zip archives holding either of these. The readable text is pulled out for indexing, and the document is assigned a random 128-bit identifier that carries no information about its content.

Step 2 – Content and Filename Encryption: The raw bytes of the file and its name are encrypted under K_{file}, each with its own independently chosen nonce. The cloud receives five items per document (identifier, nonce, ciphertext, tag and encrypted filename) and stores them as one row of its documents table.

3.2.1 Numerical Example of AES-256-GCM Encryption

To show what equation (4) does at the level of individual bytes, we traced one complete AES-256-GCM encryption by hand and checked every intermediate value with a small script that implements AES and GCM from scratch. The final ciphertext and tag produced by that script are identical to the output of the AESGCM class of the cryptography library used in our system, and the underlying AES-256 block cipher reproduces the published FIPS-197 known-answer value for this key. In the real system K_{file} comes from HKDF-SHA256 and the nonce from the operating system's random source; for this example we fixed both so that the numbers can be reproduced. All values are in hexadecimal, and no associated data is used.

Step 1 – Inputs: The 256-bit key, 96-bit nonce and 16-byte plaintext used in the example are listed in Table I. The plaintext is the ASCII string "Encrypted vault!", which is exactly one 128-bit AES block.

Item	Value (hex)	Size
K_file (example)	00010203 04050607 08090a0b 0c0d0e0f 10111213 14151617 18191a1b 1c1d1e1f	32 B
Nonce N	cafebabe facedbad decaf888	12 B
Plaintext M = "Encrypted vault!"	456e6372 79707465 64207661 756c7421	16 B

Table I Inputs of the worked example

Step 2 – Key Expansion: AES-256 treats the key as eight 32-bit words w_0 to w_7 and expands them into 60 words, which form 15 round keys (one for the initial AddRoundKey and one for each of the 14 rounds). Every eighth word is produced with RotWord, SubWord and a round constant; for example, the first new word is computed from w_7 = 1c1d1e1f as follows:

$$RotWord(1c1d1e1f) = 1d1e1f1c, \quad SubWord(1d1e1f1c) = a472c09c \tag{4a}$$

$$w_8 = w_0 \oplus (a472c09c \oplus 01000000) = 00010203 \oplus a572c09c = a573c29f \tag{4b}$$

Words w_i with i mod 8 = 4 pass through SubWord only, and all other words are the XOR of w_{i-1} and w_{i-8}. Table II lists the resulting round keys.

Round	Round key (hex)
0	00010203 04050607 08090a0b 0c0d0e0f
1	10111213 14151617 18191a1b 1c1d1e1f
2	a573c29f a176c498 a97fce93 a572c09c
3	1651a8cd 0244beda 1a5da4c1 0640bade
4	ae87dff0 0ff11b68 a68ed5fb 03fc1567
5	6de1f148 6fa54f92 75f8eb53 73b8518d
6	c656827f c9a79917 6f294cec 6cd5598b
7	3de23a75 524775e7 27bf9eb4 5407cf39
8	0bdc905f c27b0948 ad5245a4 c1871c2f
9	45f5a660 17b2d387 300d4d33 640a820a
10	7ccff71c beb4fe54 13e6bbf0 d261a7df
11	f01afafe e7a82979 d7a5644a b3afe640
12	2541fe71 9bf50025 8813bbd5 5a721c0a
13	4e5a6699 a9f24fe0 7e572baa cdf8cdea
14	24fc79cc bf0979e9 371ac23c 6d68de36

Table II AES-256 round keys derived from the example key

Step 3 – Counter Blocks and Hash Key: GCM does not encrypt the plaintext with AES directly. It first builds counter blocks from the 96-bit nonce. The pre-counter block J_0 is the nonce followed by the 32-bit value 1, and the first counter block used for data, CB_1, is obtained by incrementing the last 32 bits. The hash key H used later for authentication is the AES encryption of an all-zero block:

$$J_0 = N \parallel 00000001 = cafebabe\ facedbad\ decaf888\ 00000001 \tag{4c}$$

$$CB_1 = inc_{32}(J_0) = cafebabe\ facedbad\ decaf888\ 00000002 \tag{4d}$$

$$H = AES_K(0^{128}) = f29000b6\ 2a499fd0\ a9f39a6a\ dd2e7780 \tag{4e}$$

Step 4 – AES-256 Rounds on CB_1: The 16 bytes of CB_1 are arranged column by column into a 4 × 4 state matrix. The initial round only XORs the state with RK_0 (AddRoundKey). For example, the first byte becomes ca ⊕ 00 = ca and the second becomes fe ⊕ 01 = ff.

CB1 (input)				⊕	RK0				=	State after Round 0			
ca	fa	de	00		00	04	08	0c		ca	fe	d6	0c
fe	ce	ca	00		01	05	09	0d		ff	cb	c3	0d
ba	db	f8	00		02	06	0a	0e		b8	dd	f2	0e
be	ad	88	02		03	07	0b	0f		bd	aa	83	0d

Table III Round 0: AddRoundKey

Rounds 1 to 13 each apply four operations. SubBytes replaces every byte using the AES S-box, so the first byte ca becomes S(ca) = 74. ShiftRows rotates row r of the state left by r positions. MixColumns multiplies each column by a fixed matrix over $GF(2^8)$; for the first column [74, 1f, 89, d7] the first output byte is

$$(02 \cdot 74) \oplus (03 \cdot 1f) \oplus 89 \oplus d7 = e8 \oplus 21 \oplus 89 \oplus d7 = 97 \tag{4f}$$

and AddRoundKey then XORs the result with the round key, giving 97 ⊕ 10 = 87. Tables III to V show Round 0 and Round 1 in full.

State in				→	SubBytes				→	ShiftRows			
ca	fe	d6	0c		74	bb	f6	fe		74	bb	f6	fe
ff	cb	c3	0d		16	1f	2e	d7		1f	2e	d7	16
b8	dd	f2	0e		6c	c1	89	ab		89	ab	6c	c1
bd	aa	83	0d		7a	ac	ec	d7		d7	7a	ac	ec

Table IV Round 1: SubBytes and ShiftRows

MixColumns				⊕	RK1				=	State after Round 1			
97	ce	55	f0		10	14	18	1c		87	da	4d	ec
1d	7b	5b	66		11	15	19	1d		0c	6e	42	7b
00	56	16	5e		12	16	1a	1e		12	40	0c	40
bf	a7	f9	0d		13	17	1b	1f		ac	b0	e2	12

Table V Round 1: MixColumns and AddRoundKey

The same four operations are repeated in Rounds 2 to 13. Round 14 omits MixColumns, as specified for the final AES round. Table VI gives the state at the end of every round; the output of Round 14 is the keystream block $AES_K(CB_1)$.

Round	State at end of round (hex)
1	870c12ac da6e40b0 4d420ce2 ec7b4012
2	0691ed4e e3f3057d 3979b790 aad85109
3	77c580da 1f65d1d0 378d6726 d628e11b
4	a2d39c79 dccf9438 68406a57 cce83b0b
5	b5f938d8 37bade9f 90ee9bd7 3602d5a2
6	5e76c58f fc098e4b ea752391 c5d0ed1c
7	344e9099 b35a6702 b79af08d d37bfbf0
8	e2a0ba46 2a82eb8f 906e94eb a476a395
9	530717b6 2c365392 3f6973b9 cd51c211
10	bf2087a5 270dee87 2e6e35d0 5c37e487
11	03112b89 b1fca2cc 0095d8ba bd98dbe1
12	811ae4c6 701e79a0 3ebfba51 fdad97cf
13	be3f6e04 2f84dae0 2c8fe540 ffe324bb
14	8aa3a026 aa7a4f1b 460b5ddd 7b1d893f

Table VI State after each of the 14 AES-256 rounds applied to CB1

Step 5 – Producing the Ciphertext: The ciphertext is the XOR of the plaintext with this keystream. For the first byte, the letter "E" (45) gives 45 ⊕ 8a = cf.

$$AES_K(CB_1) = 8aa3a026\ aa7a4f1b\ 460b5ddd\ 7b1d893f \tag{4g}$$

$$CT = M \oplus AES_K(CB_1) = cfcdc354\ d30a3b7e\ 222b2bbc\ 0e71fd1e \tag{4h}$$

Step 6 – Computing the Authentication Tag: GCM authenticates the ciphertext with GHASH, a polynomial evaluation in GF(2^{128}) keyed by H. With one ciphertext block and no associated data, GHASH takes two multiplications. The length block L holds the bit lengths of the associated data (0) and of the ciphertext (128 = 0x80). The tag is the GHASH result masked with the encryption of J_0:

$$X_1 = CT \cdot H = 261a97f6\ b30e9a94\ 64f30d80\ ed3ae0b4 \tag{4i}$$

$$S = (X_1 \oplus L) \cdot H = e6dc53d5\ 5ceb0a83\ 2b70d8a9\ 5d76cd0e \tag{4j}$$

$$AES_K(J_0) = 5b0a77e3\ 716cc7e8\ 6cf71b26\ dee7d5a4 \tag{4k}$$

$$T = AES_K(J_0) \oplus S = bdd62436\ 2d87cd6b\ 4787c38f\ 839118aa \tag{4l}$$

Step 7 – Stored Output: The client sends the cloud the nonce, the ciphertext and the tag, (N, CT, T). For the 16-byte plaintext this is 44 bytes in total (12 + 16 + 16), which is exactly 28 bytes more than the plaintext, as stated in Section 3.2 and measured in Section IV.

3.2.2 Numerical Example of AES-256-GCM Decryption

Decryption in GCM uses the same forward AES cipher as encryption. Because the plaintext is recovered by XOR with a keystream, the inverse AES operations (InvSubBytes, InvShiftRows and InvMixColumns) are never needed. The client recomputes the keystream and the tag from the received data, and the tag is checked before any plaintext is returned.

Step 1 – Splitting the Stored Record: The record fetched from the cloud is split into N = cafebabe facedbad decaf888, CT = cfcdc354 d30a3b7e 222b2bbc 0e71fd1e and T = bdd62436 2d87cd6b 4787c38f 839118aa.

Step 2 – Tag Verification: Using K_{file}, the client recomputes H, J_0 and GHASH over the received CT exactly as in Steps 3 and 6, obtaining T′ = bdd62436 2d87cd6b 4787c38f 839118aa. Since T′ = T, the ciphertext is accepted as unmodified.

Step 3 – Keystream Regeneration: The client runs the same 14 rounds on CB_1 = cafebabe facedbad decaf888 00000002 (Tables III–VI) and obtains the same keystream 8aa3a026 aa7a4f1b 460b5ddd 7b1d893f.

Step 4 – Recovering the Plaintext: XORing the ciphertext with the keystream reverses Step 5 of encryption, because (M ⊕ K) ⊕ K = M. For the first byte, cf ⊕ 8a = 45, which is the letter "E".

$$M = CT \oplus AES_K(CB_1) = 456e6372\ 79707465\ 64207661\ 756c7421 \quad (4m)$$

Decoding these bytes as ASCII gives back the original text "Encrypted vault!".

Step 5 – Effect of Tampering: To check the integrity protection, we flipped one bit of the stored ciphertext, changing its first byte from cf to ce. Recomputing the tag over the modified ciphertext gives T′ = bd5dd193 9f6b1a32 6802ee5b 085f8f84, which does not match the stored T. The library rejects the record, and our client reports its single generic decryption error without releasing any plaintext, which is the behaviour described in Section 3.2 and in equation (10).

3.3 Keyword Extraction and Encrypted Index Construction

The encrypted index is what makes the vault searchable. All keyword extraction goes through one deterministic tokenizer. It converts text to lower case, splits it into alphanumeric tokens, discards stop words and anything shorter than two characters, and keeps each remaining word only once per document. Queries pass through the very same function, so a word cannot be normalised one way during indexing and another way during search; a query for "Encryption." therefore finds documents containing "encryption". Every keyword w gets two keys of its own, derived from K_{index} with the pseudorandom function F = HMAC-SHA256:

$$K_1 = F(K_{index}, \text{“1”} \Vert w), \quad K_2 = F(K_{index}, \text{“2”} \Vert w) \quad (5)$$

K_1 is used to produce lookup labels and K_2 to encrypt document identifiers. Let DB(w) be the list of identifiers of documents in which w occurs. Each (keyword, document) pair, which we refer to as a posting, contributes one entry to the index:

$$\ell_c = F(K_1, c), \quad v_c = AES\text{-}GCM(K_2, id_c), \quad c = 0, 1, \ldots, |DB(w)| - 1 \quad (6)$$

The counter c runs separately for each keyword. ℓ_c is a 256-bit value that we store as a 64-character hexadecimal string, and v_c is the encrypted identifier of the c-th document containing w. Physically, the index is nothing more than a flat table of (label, value) rows. It holds no keywords, no identifiers in the clear, and nothing that groups the rows of one keyword together. Documents are processed in sorted order so that the labels are reproducible, but each value is encrypted under a fresh nonce, so an unchanged posting still produces a different ciphertext after a rebuild.

Step 1 – Corpus Tokenisation: The client decrypts every document in the vault and runs the tokenizer over it, giving a map from document identifier to keyword set. Grouping this map by keyword yields DB(w) for each keyword.

Step 2 – Index Generation and Atomic Replacement: For each keyword the client derives K_1 and K_2, writes one label–value pair per posting, and uploads the finished index. The cloud swaps the old index for the new one inside a single transaction. Because the construction is static, any upload, deletion or password change requires the entire index to be rebuilt; Section IV measures what this costs.

3.4 Search Token Generation and Encrypted Search

A search begins with the client passing the query through the same tokenizer and deriving K_1 for the keyword. K_1 is the only key material that ever leaves the trusted side. K_2 is computed on the client only when it is needed and is never transmitted, since the cloud has no reason to decrypt anything. Using the token, the cloud computes the labels $\ell_0, \ell_1, \ldots$ one after another and checks each against the index, stopping at the first counter value that has no matching label. This tells the cloud which opaque rows exist, but not the keyword, the identifiers or any document content. The cloud carries its own HMAC implementation written with the standard library, and the cloud package imports nothing from the client package. A conjunctive query over q keywords is handled as q separate single-keyword searches, whose decrypted results the client intersects:

$$R(w_1 \wedge w_2 \wedge \ldots \wedge w_q) = R(w_1) \cap R(w_2) \cap \ldots \cap R(w_q) \quad (7)$$

R(w) stands for the set of identifiers of documents matching w. The intersection is computed on the trusted client and ends as soon as the running result becomes empty. Supporting AND queries this way did not require any change to the cloud code or the index format. The drawback is additional leakage: the cloud sees the result set of each keyword separately and can tell that these keywords belonged to the same query. Section 3.6 lists this co-occurrence leakage explicitly.

Step 1 – Token Generation: After tokenising the query, the client derives K_1 for every keyword and sends each one to the cloud in a search request that contains nothing else.

Step 2 – Cloud Search and Client-Side Decryption: The cloud executes the label regeneration loop and sends back the encrypted values it locates. On the client, K_2 is derived and used to decrypt the identifiers, and for multi-keyword queries the resulting sets are intersected. To show the results, the client also downloads each matching document's encrypted filename and decrypts it, and any selected document can be downloaded and decrypted with K_{file}.

3.5 Document Deletion and Password Rotation

The order of operations during deletion matters. The client first rebuilds the encrypted index without the document in question and replaces the cloud's copy; only after that does it ask the cloud to remove the ciphertext. Doing it the other way round would be risky, because a failed rebuild would leave labels in the index pointing at a document that had already been removed.

Every key in the hierarchy is derived from the password, so changing the password means re-encrypting everything. Before starting, the client independently derives keys from the current password and checks them against the stored verifier. The rotation then runs in two phases. During validation, the client fetches, decrypts and tokenises the whole corpus with the old keys but writes nothing; a failure on even one document aborts the rotation and leaves the vault exactly as it was. Only then does the write phase begin. Fresh keys are derived from a newly generated salt, the documents are re-encrypted in place one by one, a new index is produced with the new index key, and the salt and verifier are replaced last of all.

Step 1 – Validate Before Writing: The rebuild after an upload, deletion and rotation could each leave the vault partly updated if interrupted. All three therefore check the full affected corpus first and make no writes unless every document can be processed.

Step 2 – Documented Limitation: Rotation does not update all documents as one atomic unit. Should the client lose its connection to the cloud midway, some documents may already use the new keys while others

still use the old ones. Rather than hide this, the system reports it to the user and we document it; making rotation atomic through envelope encryption is part of our future work.

3.6 Trust Boundary and Leakage Model

No efficient SSE scheme hides everything from the server, so we wrote down exactly what our cloud can learn and checked each item with automated tests that record the real HTTP traffic between the two processes. Several things never reach the cloud at all: the password, either sub-key, any keyword key K_2, and any plaintext content, filename or keyword. The cloud can observe how many documents are stored and the size of each ciphertext. The row count of the index is also visible, and this equals the total number of postings. Because a given keyword always yields the same token K_1, it can tell when a search is repeated (the search pattern). It sees which index rows match each token and which documents are then fetched by identifier (the access pattern). For conjunctive queries it also sees the result set of each keyword and the fact that those keywords were queried together.

The static index adds one more observable event: every upload or deletion triggers a rebuild during which the client downloads all stored documents, and the cloud can see this happen. We accept these leakages in exchange for efficient search and report them as limitations of the system rather than claiming they do not exist. The complete architecture is given in Fig. 1.

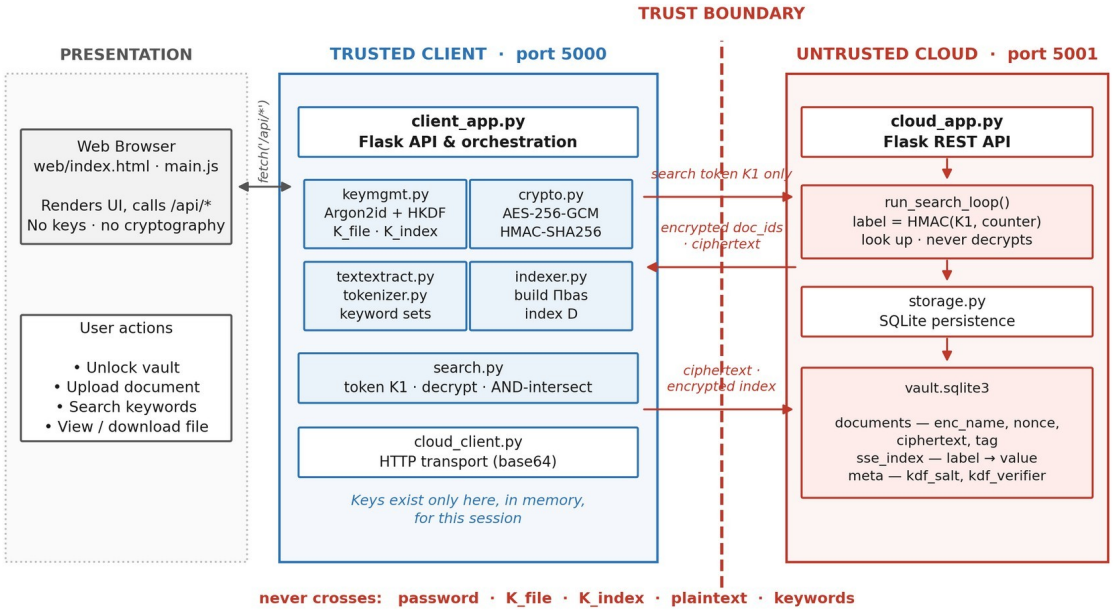


Fig. 1 Proposed SSE-based encrypted cloud storage system with a process-level trust boundary

3.7 SSE-Based Encrypted Storage and Search Algorithm

Putting the modules together, the algorithm keeps documents on an untrusted server and finds them by keyword without revealing the documents, their filenames or the keywords. Key management uses Argon2id and HKDF, documents are protected with AES-256-GCM, and search runs over an encrypted inverted index built from HMAC. A query costs the client one short token per keyword plus the decryption of the matching identifiers, instead of a download and decryption of the entire vault. Fig. 2 shows the full workflow.

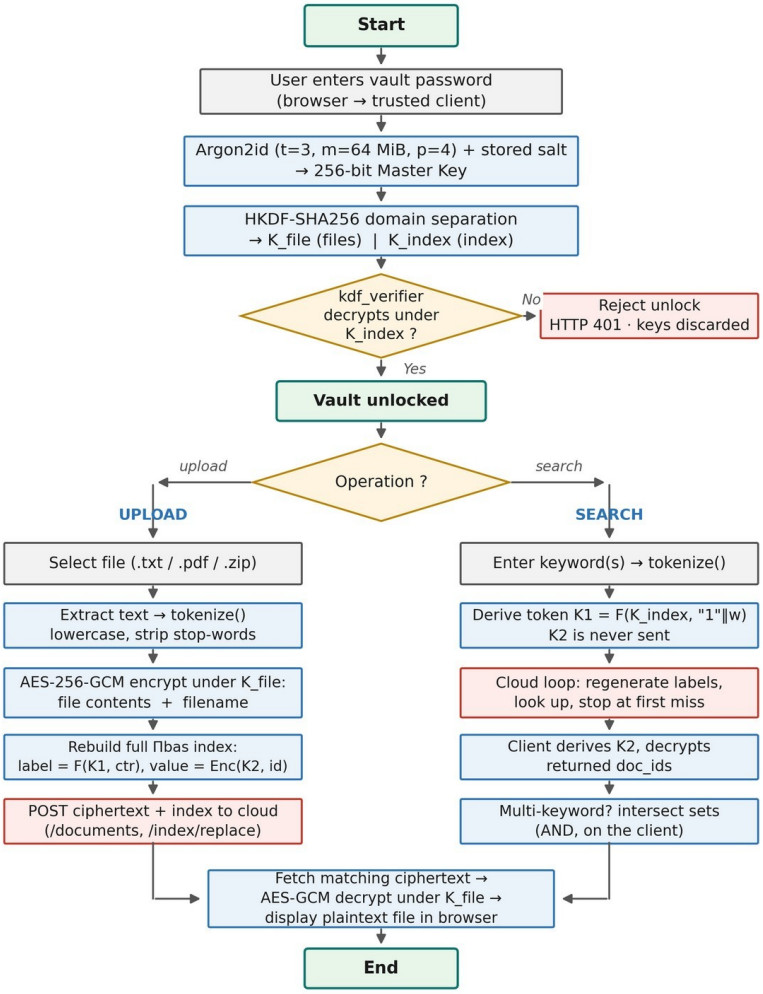


Fig. 2 Complete workflow of the implemented system

Input: Password pw, document set $D = \{D_1, D_2, \ldots, D_n\}$ and query keywords $w_1, \ldots, w_q$

Output: Encrypted documents and encrypted index stored on the cloud, followed by the decrypted documents matching the query

Step 1 - Derive the keys

Argon2id combines the password with the stored salt to give the master key, from which HKDF-SHA256 derives K_{file} and K_{index} according to (1) and (2). Decrypting the verifier from (3) confirms that the password was entered correctly.

Step 2 - Extract and tokenise each document

For each uploaded document, the readable text is extracted and run through the deterministic tokenizer, producing its keyword set $W(D_i)$.

Step 3 - Encrypt the document and its filename

The document receives a random identifier id_i, and its contents and filename are encrypted under K_{file} with fresh nonces according to (4).

Step 4 - Upload the encrypted document

Only the identifier, encrypted filename, nonce, ciphertext and tag are transmitted to the cloud. Neither the plaintext nor any key leaves the client.

Step 5 - Derive the keyword keys

Using (5), K_1 and K_2 are computed from K_{index} for each keyword w that appears in the corpus.

Step 6 - Build the encrypted index

Each posting of w is turned into a label and an encrypted value using (6). The resulting table of (label, value) pairs exposes no keywords and no identifiers.

Step 7 - Replace the index on the cloud

The cloud receives the whole index and installs it in one transaction, so at no point does it hold an index that is only partly built.

Step 8 - Generate the search token

The client normalises each query keyword and transmits only $T_w = K_1$, keeping K_2 to itself.

Step 9 - Run the cloud search loop

Starting from the token, the cloud regenerates labels and collects the stored values that match, stopping at the first label not present in the index I:

$$S(w) = \{ v_c : \ell_c = F(T_w, c) \in I,\ c = 0, 1, 2, \dots \} \tag{8}$$

Step 10 - Decrypt the matching identifiers

With K_2 derived locally, the client decrypts each returned value to obtain the set of matching identifiers:

$$R(w) = \{ AES\text{-}GCM^{-1}(K_2, v) : v \in S(w) \} \tag{9}$$

Step 11 - Combine multi-keyword results

When the query contains several keywords, the client intersects their identifier sets using (7).

Step 12 - Retrieve and decrypt the documents

The client fetches the matching encrypted documents by identifier and decrypts them with K_{file}. Because the tag is verified during decryption, a ciphertext that has been modified is rejected instead of being returned to the user.

$$M = AES\text{-}GCM^{-1}(K_{file}, N, CT, T) \tag{10}$$

IV. RESULT AND DISCUSSION

4.1: Simulation Tools

The implementation is written in Python 3.13, and the two web services run on Flask 3.1.3. AES-256-GCM, HMAC-SHA256 and HKDF-SHA256 come from version 50.0.1 of the cryptography library, Argon2id from argon2-cffi 25.1.0, and PDF text extraction from pypdf. The cloud keeps its data in SQLite 3.50.4, and the automated tests are run with pytest.

Functional behaviour was checked with a small corpus of eight sample documents that we wrote by hand. All performance figures come from a Windows 11 laptop with a 64-bit Intel processor (10 physical and 12 logical cores) and 16 GB of RAM. For benchmarking we generated a synthetic corpus deterministically,

with Zipf-like word frequencies and roughly 40% stop words. Because the generator is deterministic, smaller corpora are simply the first documents of the larger ones (the 10-document set is the start of the 200-document set), so repeated runs always work on identical text. Each timing point was measured n = 10 to 30 times after a warm-up, and the median is reported. The evaluation used the following parameters:

Correctness: Search results must agree exactly with the ground truth of the corpus, and retrieved documents must decrypt successfully

Search Latency: Time spent by the cloud in its search loop, and end-to-end search time measured at the client

Processing Time: Time taken for key derivation, encryption, decryption, index construction and index rebuilding

Storage Overhead: Size of the encrypted database compared with the size of the plaintext

Key Size: AES-256-GCM with 256-bit keys, HMAC-SHA256 with 256-bit labels, Argon2id with a 64 MiB memory cost

4.2: Functional Results

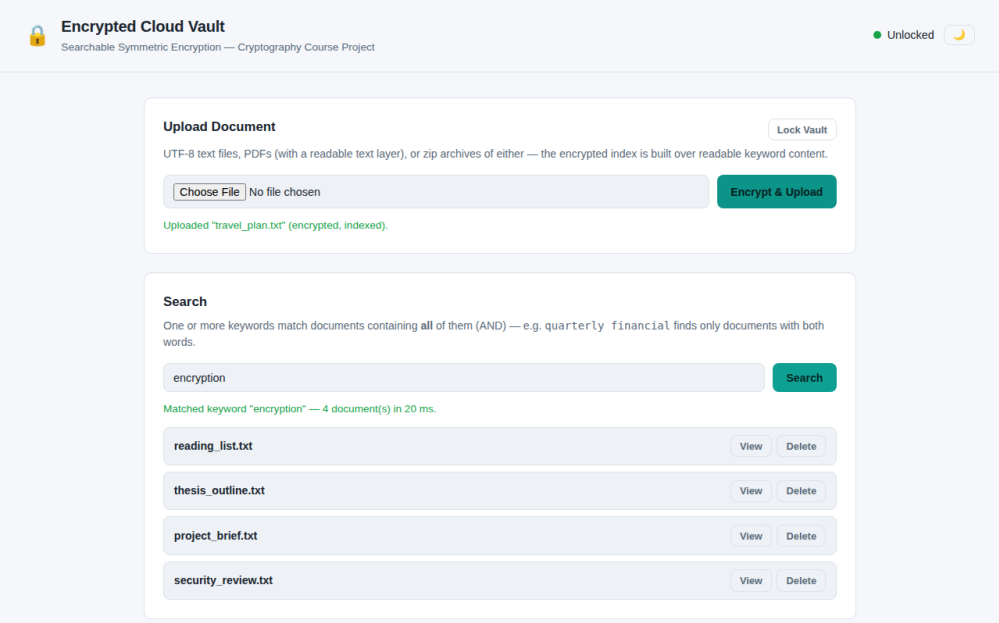


Fig. 3 Single-keyword encrypted search returning four matching documents

Our first functional check was a search for the single keyword “encryption” across the eight encrypted sample documents, whose index holds 400 postings (Fig. 3). Four documents contain this keyword, and the search returned exactly those four in 20 ms. We obtained the expected answer independently by running the tokenizer over the plaintext files, and the two agreed.

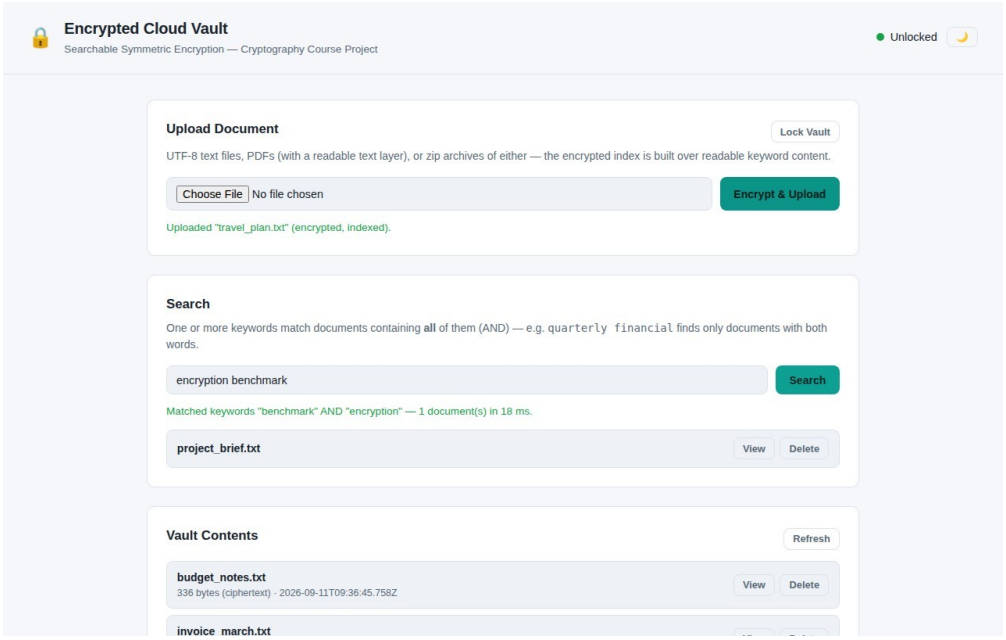


Fig. 4 Conjunctive (AND) search intersecting two keyword result sets on the client

For conjunctive search we used the query “encryption benchmark” (Fig. 4). Taken separately, each of the two keywords matches four documents. After intersecting the two result sets, the client returned the one document containing both words in 18 ms. The recorded traffic for this query contains two separate search requests carrying one token each, which agrees with the behaviour described in Section 3.4.

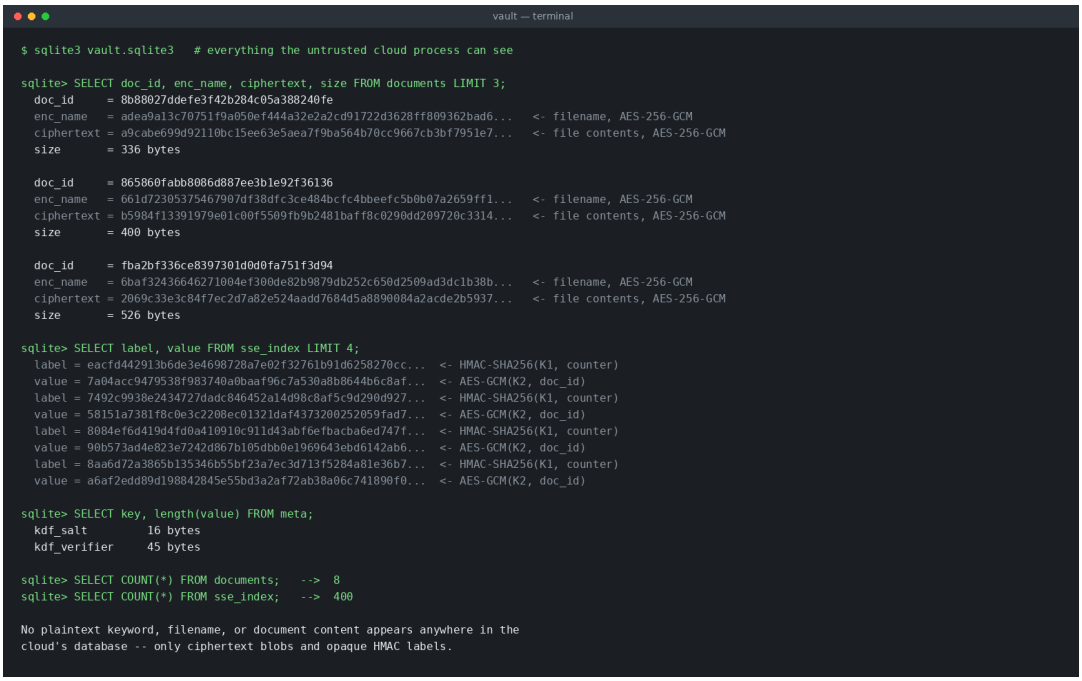


Fig. 5 Contents of the cloud database: only ciphertext and opaque HMAC labels are stored

To see the server’s view directly, we opened the cloud’s database (Fig. 5). It contains no readable filename, keyword or document text. All it holds is the password verifier, the salt (which does not need to be secret), encrypted identifiers, HMAC labels 64 characters long and AES-GCM ciphertext, which matches the trust boundary set out in Section 3.6.

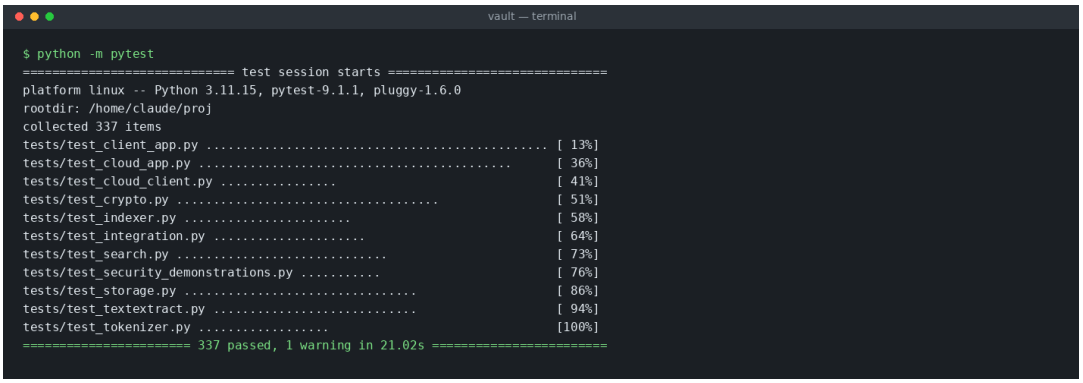


Fig. 6 Complete automated test suite passing (337 tests)

All 337 tests in the automated suite pass (Fig. 6). They exercise the cryptographic primitives, the tokenizer and the index, the search protocol, both web services, deletion and password rotation, together with security demonstrations that record genuine client–cloud traffic to check the leakage model.

4.3: Performance Evaluation

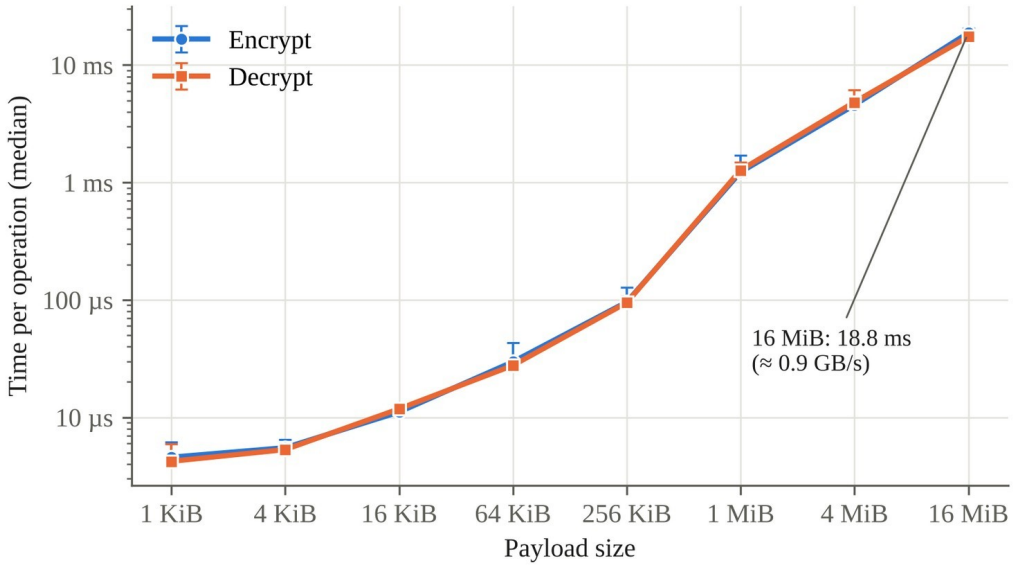


Fig. 7 AES-256-GCM encryption and decryption time versus payload size (median, min–max bars, n = 15)

The AES-256-GCM timings in Fig. 7 show encryption and decryption taking almost the same time, with both growing linearly in the amount of data. The median encryption time was 1.23 ms at 1 MiB and 18.8 ms at 16 MiB, which corresponds to about 0.9 GB/s. In every case the ciphertext was exactly 28 bytes longer than the plaintext. From these numbers, symmetric encryption is not a limiting factor in the system. The most expensive cryptographic operation is key derivation, and that is by design: a full Argon2id unlock has a median of 57 ms. For comparison, one HMAC evaluation costs about 3.4 µs, and producing a complete search token about 5 µs.

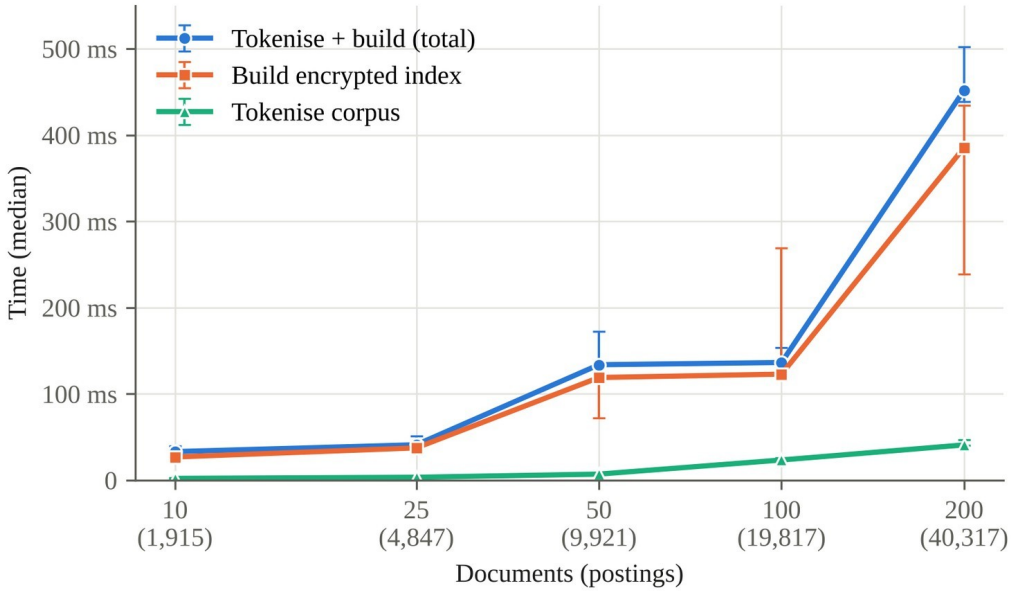


Fig. 8 Encrypted index construction time versus number of documents (median, min–max bars, n = 10)

Index construction on the client is plotted in Fig. 8. With 10 documents (1,915 postings), tokenising and building the index took 33 ms combined. With 200 documents, which produce 40,317 postings across 6,074 distinct keywords, the total was 452 ms; index construction accounted for 386 ms of this and tokenisation for only 41 ms. What drives the cost is how many postings there are, not how many documents, at around 11 µs per posting for the 200-document corpus. That is expected, since every posting costs the client an HMAC computation for its label and an AES-GCM encryption for its value. The curve has some irregularity: at 50 and 100 documents the medians are close together and the min–max ranges are wide, so we treat these middle points as approximate.

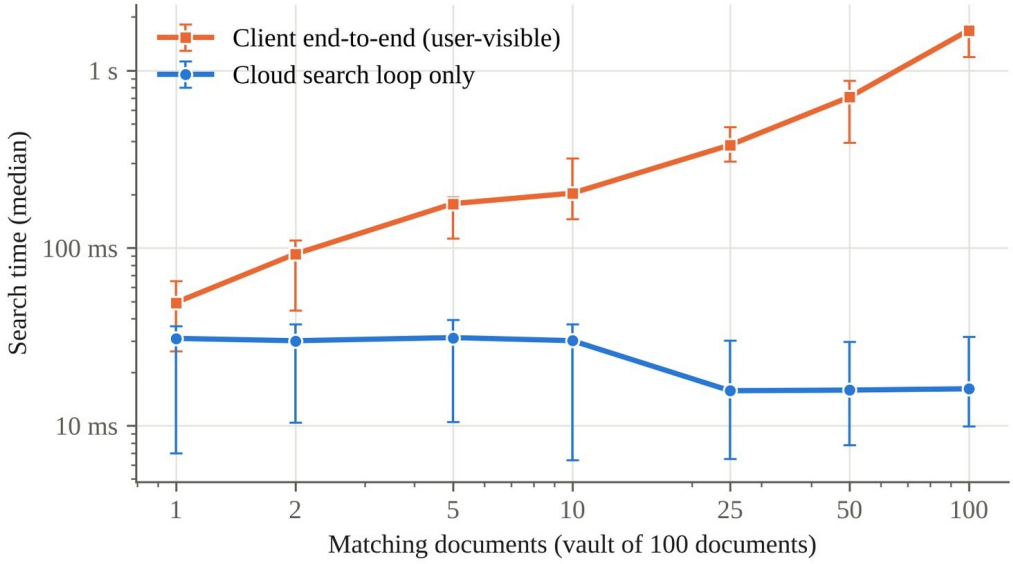


Fig. 9 Cloud-side search time and client end-to-end search latency versus number of matching documents

Fig. 9 separates the time the cloud spends in its search loop from the total latency seen by the user, for a vault of 100 documents. As the number of matching documents rises from 1 to 100, the cloud-side time stays within a roughly flat band of 16–31 ms. End-to-end latency behaves very differently, climbing almost linearly from

49 ms with one match to 1.68 s with 100. Looking at the recorded traffic showed why: the client issues one document request per result to obtain each encrypted filename for display. These per-result fetches, not the cryptographic search, form the bottleneck, and batching them is the most effective optimisation available.

In another test we fixed the number of matches at five and increased the vault from 25 to 200 documents. Cloud search time stayed between 15 and 17 ms and end-to-end time between 96 and 98 ms, so the size of the vault does not affect the search cost.

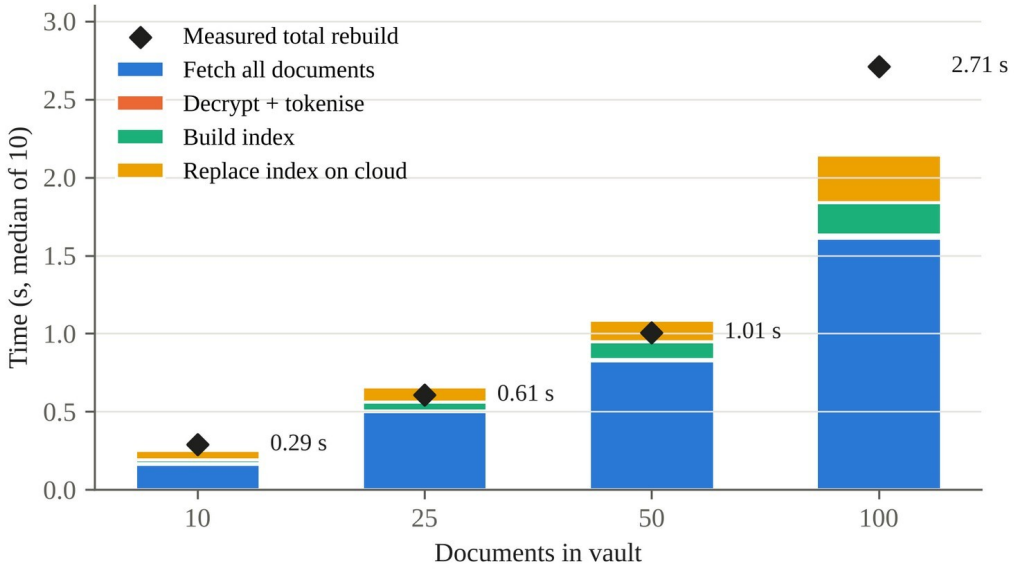


Fig. 10 Decomposition of the static index rebuild time performed after every upload

Fig. 10 splits the index rebuild that follows each upload into its component steps. The median rebuild time increases from 0.29 s for a 10-document vault to 2.71 s for a 100-document vault. At 100 documents, the individual steps add up to 2.15 s, and 1.61 s of that is spent downloading every stored document from the cloud. The other steps are much smaller: 22 ms to decrypt and tokenise, 208 ms to build the index and 305 ms to replace it on the cloud. Rebuilding is therefore dominated by data transfer between client and cloud, and its cost rises with the size of the vault; we consider this the main practical weakness of using a static index. We have no rebuild figure for 200 documents, since that checkpoint was not reached in any of our three benchmark runs.

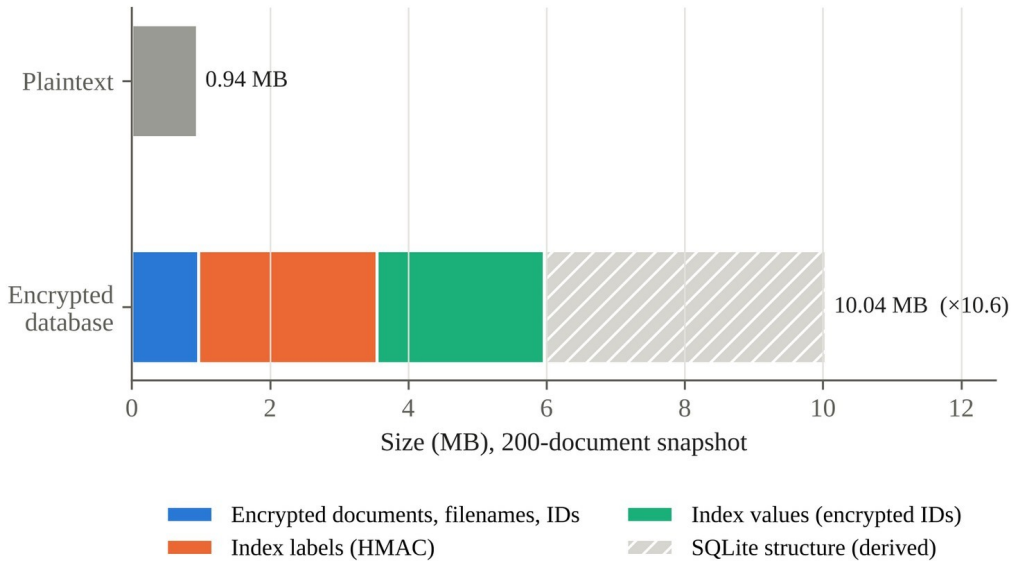


Fig. 11 Storage footprint of the encrypted database at 200 documents (single snapshot)

Fig. 11 breaks down storage use for a single snapshot of the 200-document vault. The plaintext contents and filenames together take 944,350 bytes, while the encrypted SQLite database takes 10,039,296 bytes, about 10.6 times as much. Encrypting the documents themselves adds only 28 bytes each. Most of the increase has two causes. The encrypted index needs about 5.0 MB for its 40,317 postings, or 124 bytes per posting (a 64-character label plus a 60-byte encrypted identifier). Another 4.1 MB or so is database structural overhead, which we calculated by subtracting the measured column data from the total file size. Making the data searchable therefore has a substantial storage cost, and that cost depends on the number of postings rather than the size of the documents. Because storage was measured at only one corpus size, this figure should not be read as a scaling result.

V. CONCLUSION

This project produced a working encrypted cloud storage system based on searchable symmetric encryption, with the trusted client and the untrusted cloud running as separate processes so that only ciphertext, opaque labels and per-keyword search tokens pass between them. Keys come from the user's password through Argon2id and HKDF, and the system offers AES-256-GCM encryption of documents and filenames, a static encrypted index built with HMAC, single- and multi-keyword search, deletion and password rotation, all covered by 337 passing automated tests. Our measurements show that the cryptographic operations are inexpensive and that the cloud's search time barely changes whether a query matches one document or a hundred, or whether the vault is small or large. The delays a user actually experiences come from elsewhere: per-result document requests during search, and the full static index rebuild during updates. The encrypted index also enlarges storage by about an order of magnitude. As next steps, we would fetch search results in batches, switch to a dynamic SSE scheme that provides forward and backward privacy, look for ways to leak less about the search and access patterns, use envelope encryption so that a password change either completes fully or not at all, and allow more than one authorised user.

References

[1] Z. Hu, J. Wang, Z. Chen, Z. Ge, M. Bian, L. Chen, and Y. Zhou, "SEAC: Dynamic Searchable Symmetric Encryption with Lightweight Update-Search Permission Control," Cybersecurity, vol. 8, article 75, 2025.

[2] S. Xu, X. Chen, Y. Guo, Y. Yang, S. Wang, S.-M. Yiu, and X. Cheng, "Forward-Secure Multi-User and Verifiable Dynamic Searchable Encryption Scheme within a Zero-Trust Environment," Future Generation Computer Systems, vol. 166, p. 107701, 2025.

[3] L. Yang, Y. Yang, D. Niyato, Z. Li et al., “Dynamic Searchable Symmetric Encryption With Efficient and Complete Access Control for Multi-User Cloud Computing,” IEEE Transactions on Mobile Computing, 2025.

[4] “Forward and Backward Private Conjunctive Dynamic Searchable Symmetric Encryption With Refined Leakage Function and Low Communication,” IEEE Transactions on Information Forensics and Security, vol. 20, pp. 8224–8236, 2025.

[5] Z. Ren, P. He, R. Yu, L. Deng, Y. Tong, and S. Xu, “Dynamic Searchable Symmetric Encryption with Efficient Conjunctive Query and Non-Interactive Real Deletion,” Journal of Network and Computer Applications, vol. 245, article 104387, 2026.

[6] X. Zhang, H. Wang, J. Lu, T. Li et al., “A Bloom Filter-Based Dynamic Symmetric Searchable Encryption Scheme over Cloud Data,” Cybersecurity, vol. 9, article 47, 2026.

[7] X. Wang, J. Gong, K. Zhang, and H. Qian, “Decentralized Multi-Client Boolean Keyword Search for Encrypted Cloud Storage,” Computer Standards & Interfaces, vol. 97, article 104127, 2026.

[8] P. Wu, J. Shen, Z. Cao, and X. Dong, “MMKFB: Multi-Client and Multi-Keyword Searchable Symmetric Encryption with Forward and Backward Privacy,” Frontiers of Computer Science, vol. 19, no. 3, article 193804, 2025.

[9] A. Wu, D. Feng, M. Zhang, J. Chi, and Y. Zhang, “Efficient Verifiable Searchable Encryption with Search and Access Pattern Privacy,” Security and Safety, vol. 4, article 2024022, 2025.

[10] A. N. Khan, A. Naveed, A. Mehmood, D. Arora, A. ur R. Khan, and J. Ali, “BloomSec: Scalable and Privacy-Preserving Searchable Encryption for Cloud Environments,” PLOS ONE, vol. 20, no. 12, article e0336944, 2025.

[11] J. Xue, L. Zeng, W. Zhang, F. Li, X. Zhang, Y. Zhou, and L. Liu, “Incentive-Assisted Multi-Keyword Ranked Searchable Encryption with Hidden Attributes for Community Data Sharing,” Peer-to-Peer Networking and Applications, vol. 18, no. 3, article 153, 2025.

[12] H. M. Mohammed, M. H. Hashim, D. Kh. Altmemi, Z. A. Abduljabbar, Z. A. Hussien, V. O. Nyangaresi, A. A. Alyousif, and H. A. Neamah, “A Review: Searchable Symmetric Encryption Functionalities, Challenges and Solutions,” in Software Engineering: Emerging Trends and Practices in System Development, Lecture Notes in Networks and Systems, vol. 1562, Springer, Cham, pp. 47–65, 2026.

[13] D. Cash, J. Jaeger, S. Jarecki, C. Jutla, H. Krawczyk, M.-C. Roşu, and M. Steiner, “Dynamic Searchable Encryption in Very-Large Databases: Data Structures and Implementation,” in Proc. Network and Distributed System Security Symposium (NDSS), 2014.

[14] A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson, “Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications,” RFC 9106, Internet Engineering Task Force, Sept. 2021.